

Elektrotehnički fakultet
Univerziteta u Beogradu



Dunja Janković 2020/0058

Razvoj interaktivnog asemblera i simulatora podskupa RISC-V instrukcija

mentor: doc. dr Uroš Radenković

Beograd, 2026.

Sadržaj

1. Uvod.....	1
2. Pregled postojećih rešenja	3
2.1. Kriterijumi izbora i poređenja	3
2.2. RARS	3
2.3. Venus	4
2.4. Poređenje sa razvijenim simulatorom.....	4
3. Teorijske osnove RISC-V arhitekture i simulacije	6
3.1. RISC-V i pojam arhitekture skupa instrukcija	6
3.2. Registri, programski brojač i memorija	6
3.3. Grupe podržanih instrukcija i pseudo-instrukcije	7
3.4. Uloga parsera, assemblera i simulatora.....	8
4. Korišćene tehnologije i razvojni alati	9
4.1. TypeScript	9
4.2. HTML, CSS i neposredan rad sa DOM-om	9
4.3. Vite i razvojno okruženje	10
4.4. Vitest i provera tipova	10
5. Projektovanje i implementacija sistema	11
5.1. Zahtevi i granice sistema	11
5.2. Arhitektura i tok podataka.....	11
5.3. Interna reprezentacija programa i stanja procesora	13
5.4. Sintaksna analiza asemblerskog koda.....	14
5.5. Dvoprolazni assembler i prevođenje pseudo-instrukcija	15
5.6. Izvršavanje instrukcija, memorija i kontrola toka	16
5.7. Kontrole izvršavanja i <i>breakpointi</i>	17
5.8. Povezivanje modela sa korisničkim interfejsom	17
6. Funkcionalnosti i primer korišćenja	19
6.1. Izgled aplikacije i režimi rada.....	19

6.2. Unos i asembliranje programa	19
6.3. Izvršavanje korak po korak.....	19
6.4. <i>Run, breakpointi i Reset</i>	20
6.5. Prikaz registara, memorije i grešaka.....	21
7. Testiranje i analiza rezultata.....	23
7.1. Strategija i organizacija testiranja.....	23
7.2. Testiranje parsera i asemblera.....	23
7.3. Testiranje modela procesora i graničnih slučajeva	23
7.4. Ručna provera korisničkog interfejsa	24
7.5. Rezultati i ograničenja provere	25
8. Zaključak.....	26

1. Uvod

Razumevanje izvršavanja programa na nivou asemblerskog jezika podrazumeva praćenje veze između instrukcija, sadržaja registara, memorije i redosleda izvršavanja. Konačan rezultat programa ne pokazuje sva međustanja kroz koja je procesor prošao da bi do njega došao. Zbog toga je za analizu rada programa značajno posmatrati ne samo izlazne vrednosti, već i promene koje nastaju izvršavanjem pojedinačnih instrukcija.

Motivacija za ovaj rad jeste izrada okruženja u kome se kratak asemblerski program može napisati, izvršiti i analizirati uz neposredan uvid u stanje simuliranog procesora. Na primeru petlje koja sabira niz brojeva, korisnik treba da može da prati promenu zbira, vrednost brojača i trenutak u kome uslov grananja prestaje da bude ispunjen. Takav način rada namenjen je povezivanju tekstualnog zapisa programa sa njegovim izvršavanjem, bez potrebe da se za svaki korak ručno vodi evidencija svih promena.

Predmet rada vezan je za RISC-V, porodicu arhitektura skupa instrukcija, odnosno pravila koja određuju instrukcije i stanje procesora vidljivo programu. Specifikacija RISC-V razdvaja osnovni celobrojni skup instrukcija od dodatnih proširenja, pri čemu RV32I označava osnovnu varijantu sa 32-bitnim celobrojnim registrima [1]. U ovom radu razmatra se odabrani podskup instrukcija te varijante, dovoljan za primere aritmetičkih i logičkih operacija, pristupa memoriji i promene toka izvršavanja.

Predmet praktičnog dela rada jeste razvoj veb aplikacije koja objedinjuje editor asemblerskog koda, parser, assembler i simulator izvršavanja. Korisnik unosi program u tekstualni editor, nakon čega aplikacija proverava njegov zapis i prevodi ga u internu reprezentaciju pogodnu za izvršavanje. Ako je program ispravan, prelazi se u režim u kome se može upravljati izvršavanjem i pratiti stanje registara i memorije.

Osnovni cilj je realizacija preglednog alata koji omogućava izvršavanje jedne instrukcije, nastavak izvršavanja do tačke prekida i izvršavanje do završetka programa. Dodatni ciljevi obuhvataju prikaz promena stanja posle korisničke akcije, prijavljivanje grešaka sa vezom ka izvornom kodu i vraćanje simuliranog procesora u početno stanje. Pri tome je potrebno sačuvati vezu između instrukcije koju simulator izvršava i odgovarajuće linije koju korisnik vidi u editoru, kako bi praćenje programa ostalo razumljivo i nakon prevođenja pseudo-instrukcija.

Obim rešenja ograničen je na 15 instrukcija i 5 pseudo-instrukcija. Podržane instrukcije obuhvataju sabiranje i oduzimanje, bitske logičke operacije i pomeranja, učitavanje i upis memorijske reči, uslovno grananje i skokove. Model stanja sadrži 32 registra širine 32 bita, programski brojač i memoriju podataka veličine 4096 bajtova, dok se pseudo-instrukcije u fazi asembliranja prevode u podržane prave instrukcije.

Simulator predstavlja model izvršavanja na nivou instrukcija, a ne model unutrašnje hardverske organizacije procesora. Asemlirani program čuva se kao niz strukturiranih instrukcija, odvojeno od memorije podataka, bez generisanja binarnog mašinskog koda. Ne modeluju se protočna obrada, keš memorija niti trajanje instrukcija u taktovima, pa prikazani rezultati opisuju promene programskog stanja, a ne performanse stvarnog procesora.

Postupak izrade zasnovan je na razdvajanju obrade izvornog koda, izvršavanja i korisničkog interfejsa. Parser izdvaja strukturu linija i operanada, a assembler proverava da li zapis odgovara podržanim instrukcijama i u dva prolaza razrešava labele i priprema program. Model procesora potom izvršava pripremljene instrukcije i menja registre, memoriju i programski brojač, nezavisno od načina na koji se te promene prikazuju korisniku.

Za implementaciju je korišćen TypeScript, dok je korisnički interfejs realizovan veb tehnologijama HTML i CSS i neposrednim radom sa objektnim modelom dokumenta, odnosno DOM-om. Vite se

koristi za razvojno okruženje i pripremu aplikacije za isporuku, a Vitest za automatizovane testove. Logika parsera, assemblera i modela procesora ne zavisi od elemenata veb stranice, što omogućava njenu proveru odvojeno od prikaza.

Interakcija sa programom organizovana je kroz režim pisanja i režim izvršavanja. U režimu izvršavanja komanda *Step* izvršava jednu instrukciju, dok *Run* nastavlja rad do završetka, tačke prekida ili greške. Prikaz izdvaja sledeću instrukciju, registre u koje je upisivano i memorijske reči koje su menjane, a *Reset* omogućava ponovno izvršavanje učitanoog programa od početnog stanja.

Prijavljivanje grešaka obuhvata i obradu izvornog teksta i samo izvršavanje. Neispravan zapis instrukcije ili nedefinisana labela sprečavaju pokretanje programa, dok nedozvoljen pristup memoriji ili neispravno odredište skoka prekidaju izvršavanje. Uvedeno je i ograničenje broja instrukcija u jednom pozivu komande *Run*, čime se ograničava trajanje izvršavanja programa koji ne dolazi do završetka ili tačke prekida.

Provera rešenja zasniva se na automatizovanim testovima komponenti i ručnoj proveru korisničkog interfejsa. Testovi obuhvataju ispravne programe i granične slučajeve, kao što su prekoračenje dozvoljenog opsega konstante, 32-bitno prelivanje, neporavnat pristup memoriji i ponašanje grananja i skokova. Rezultati ovih provera pokazuju kako se implementirane funkcije ponašaju u ispitanim slučajevima. Oni ne predstavljaju formalni dokaz ispravnosti, a na osnovu njih se ne može zaključiti koliko alat pomaže korisnicima u učenju.

Rezultat praktičnog dela jeste integrisana aplikacija u kojoj se program može pratiti od unosa izvornog teksta do promene stanja simuliranog procesora. Doprinos rada je u projektovanju, povezivanju i proveru komponenti koje ostvaruju zadate funkcionalnosti u okviru jasno ograničenog skupa instrukcija. Namenjena je demonstraciji i analizi kraćih asemblerskih programa, dok podrška celokupnom skupu instrukcija i složenijem razvojnom okruženju ostaje izvan obima ovog rešenja.

Rad je organizovan u osam poglavlja. Posle uvoda, drugo poglavlje daje pregled postojećih rešenja i kriterijuma za njihovo poređenje, dok treće objašnjava teorijske osnove potrebne za razumevanje simulatora. Četvrto poglavlje predstavlja korišćene tehnologije, a peto arhitekturu i implementacione detalje sistema.

U šestom poglavlju prikazane su funkcionalnosti aplikacije kroz primer korišćenja i praćenja izvršavanja programa. Sedmo poglavlje opisuje postupak testiranja, rezultate i ograničenja sprovedene provere. Osmo poglavlje donosi zaključak o ostvarenim ciljevima i razmatra moguća unapređenja rešenja.

2. Pregled postojećih rešenja

Za poređenje sa simulatorom razvijenim u ovom radu odabrani su alati RARS i Venus. Oba rešenja namenjena su radu sa RISC-V asemblerskim programima i omogućavaju praćenje njihovog izvršavanja [2] [4]. Pregled je usmeren na mogućnosti značajne za ovaj rad, a ne na iscrpan popis svih funkcija ili rangiranje alata.

2.1. Kriterijumi izbora i poređenja

RARS i Venus izabrani su zato što objedinjuju obradu asemblerskog programa i simulaciju, umesto da zahtevaju fizički procesor za izvršavanje. Njihova dokumentovana podrška za izvršavanje korak po korak i tačke prekida odgovara osnovnim zahtevima razvijenog rešenja [2-4]. Istovremeno predstavljaju dva različita načina korišćenja: RARS se distribuira kao Java aplikacija, dok je Venus dostupan kroz veb pregledač.

Porede se način pokretanja alata, podržani skup instrukcija, kontrole izvršavanja, prikaz registara i memorije, kao i mogućnost uporednog prikaza izvornog i mašinskog koda. Razmatraju se i dodatne mogućnosti, poput vraćanja prethodnog koraka i poziva simuliranog okruženja. Ovi kriterijumi omogućavaju da se funkcionalnosti alata povežu sa konkretnim zadacima: praćenjem petlje, analizom skoka, posmatranjem upisa u memoriju i razumevanjem prevođenja instrukcija.

Pregled je zasnovan na dokumentaciji projekata i dostupnom interfejsu Venus-a. Nisu sprovedena uporedna merenja brzine, sistematska provera usaglašenosti sa specifikacijom niti istraživanje upotrebljivosti sa korisnicima. Na osnovu ovog pregleda mogu se porediti opisane mogućnosti alata, ali se ne može oceniti njihova brzina, potvrditi potpuna ispravnost niti utvrditi koliko pomažu u učenju.

2.2. RARS

RARS, odnosno *RISC-V Assembler and Runtime Simulator*, predstavlja okruženje za asembliranje i simulaciju RISC-V programa. Distribuira se u obliku izvršne JAR datoteke, za čije pokretanje je prema uputstvu potrebna Java 8 ili novija verzija. Projekat navodi podršku za 32-bitnu i 64-bitnu varijantu arhitekture, više proširenja skupa instrukcija, pozive okruženja i asembliranje programa iz više datoteka [2].

Interaktivno izvršavanje odvija se u prikazu *Execute*. Komanda *Step* omogućava praćenje pojedinačnih instrukcija, a *Go* neprekidno izvršavanje ili nastavak nakon zaustavljanja; tačke prekida postavljaju se uz prikazane instrukcije. Dokumentacija opisuje i podešavanje brzine simulacije, pauziranje, ponovno pokretanje iz početnog stanja i prikaz registara i memorijskih vrednosti u decimalnom ili heksadecimalnom zapisu [3].

Posebno je značajna mogućnost poređenja pseudo-instrukcija sa osnovnim instrukcijama i mašinskim kodom [2]. Ona omogućava razmatranje veze između pogodnijeg zapisa programa i njegovog stvarnog kodiranja, što u razvijenom simulatoru nije predmet prikaza. RARS podržava i korake unazad, ali dokumentacija naglašava da poništavanje promena registara i memorije ne obuhvata konzolni i datotečki ulaz i izlaz [3].

U odnosu na zadati obim ovog rada, RARS obezbeđuje šire razvojno okruženje. To je relevantno za programe koji koriste dodatne instrukcije, više izvornih datoteka ili pozive okruženja, ali nije zahtev za realizaciju ovde izabranih primera. Potreba za Java okruženjem predstavlja razliku u načinu pokretanja, a ne sama po sebi nedostatak funkcionalnosti.

2.3. Venus

Venus je simulator RISC-V instrukcija namenjen obrazovanju. U ovom pregledu razmatraju se projekat kvakil/venus i veb aplikacija na koju taj projekat upućuje, bez pripisivanja mogućnosti drugih izvedenih verzija. Prema dokumentaciji, podržan je RV32IM, odnosno osnovni 32-bitni celobrojni skup instrukcija sa proširenjem za množenje i deljenje [1] [4].

Na kartici *Editor* korisnik piše program, dok na kartici *Simulator* prati njegovo izvršavanje. Za upravljanje izvršavanjem dostupne su komande *Run*, *Step*, *Prev* i *Reset*, a stanje se može pratiti u panelima za registre i memoriju. Tabela prikaz programa sadrži kolone *Machine Code*, *Basic Code* i *Original Code*, čime se omogućava povezivanje izvornog zapisa sa osnovnim instrukcijama i njihovim kodiranjem [5].

Dokumentacija navodi podršku za tačke prekida, vraćanje prethodnog koraka i određene pozive okruženja putem instrukcije *ecall*, među kojima su ispis i zahtev za dodatnim memorijskim prostorom [4]. Takvi pozivi pružaju dodatne mogućnosti programa unutar simulatora, ali ne treba ih poistovećivati sa simulacijom kompletnog operativnog sistema. Za potrebe ovog poređenja značajni su kao funkcionalnost koja nije uključena u razvijeno rešenje.

Venus je po načinu pristupa bliži aplikaciji iz ovog rada, jer se koristi kroz veb pregledač. Ipak, dostupnost gotove javne veb aplikacije treba razlikovati od lokalnog pokretanja razvijenog projekta, koje u dokumentovanom razvojnom postupku zahteva Node.js, instaliranje zavisnosti i Vite server. Sličnost platforme zato ne znači da su postupci pripreme i obim podržanih programa jednaki.

2.4. Poređenje sa razvijenim simulatorom

Jedna važna razlika odnosi se na to šta korisnik može da posmatra nakon asembliranja. RARS i Venus prikazuju i mašinski kod, pa se u njima može analizirati veza između tekstualne instrukcije i njenog kodiranja [2] [4]. Razvijeno rešenje zadržava se na internoj reprezentaciji i promeni stanja, zbog čega nije zamena za ove alate kada je cilj vežbe proučavanje bitskih polja instrukcija.

Druga razlika je način ponavljanja analize. Korak unazad u postojećim alatima omogućava povratak na prethodno stanje u okviru podržanih ograničenja [3] [4]. Razvijeni simulator ne čuva istoriju stanja: *Reset* vraća program na početak, a do željenog mesta ponovo se dolazi pojedinačnim koracima ili tačkom prekida. Ova razlika je važna kada korisnik želi da ponovi samo jednu instrukciju, umesto celog dela programa koji joj prethodi.

Razlike u podržanim instrukcijama i pozivima okruženja utiču i na prenosivost primera. Program koji u postojećem alatu koristi ispis preko *ecall* ili množenje ne može se bez prilagođavanja izvršiti u razvijenom simulatoru. Pri izboru primera zato je potrebno proveriti da li sve upotrebljene instrukcije i oblici zapisa pripadaju podržanom podskupu, a ne osloniti se samo na zajedničku oznaku RISC-V.

Sažetak odabranih mogućnosti prikazan je u Tabeli 1. Podaci za RARS i Venus preuzeti su iz navedenih izvora [2-5], dok kolona razvijenog simulatora opisuje implementaciju ovog rada. Oznaka da neka funkcija nije podržana odnosi se na trenutnu realizaciju, a ne na mogućnost njenog naknadnog dodavanja.

Tabela 1. Poređenje odabranih mogućnosti simulatora

Kriterijum	RARS	Venus	Razvijeni simulator
Okruženje	Java aplikacija	Veb aplikacija	Veb aplikacija
Skup instrukcija	32/64-bitni; više proširenja	RV32IM	15 instrukcija iz RV32I
Praćenje izvršavanja	<i>Step, Go</i> i tačke prekida	<i>Step, Run</i> i tačke prekida	<i>Step, Run</i> i tačke prekida
Registri i memorija	Prikaz stanja	Prikaz stanja	Prikaz upisivanih registara i reči
Mašinski kod	Prikazuje se	Prikazuje se	Ne generiše se
Korak unazad	Podrжан uz ograničenja	Podrжан	Nije podrжан
Pozivi okruženja	Podržani	Podržani	Nisu podržani

Poređenje pokazuje da izvršavanje korak po korak, tačke prekida i prikaz stanja već postoje u posmatranim alatima. Doprinos ovog rada zato nije uvođenje tih mogućnosti kao novih, već njihova samostalna realizacija i povezivanje parsera, assemblera, modela procesora i korisničkog interfejsa. Izostavljanje mašinskog kodiranja i dodatnih instrukcija ograničava vrste programa i pojava koje se mogu prikazati.

Razvijeni simulator usmeren je na praćenje kraćih programa u okviru 15 instrukcija i 5 pseudo-instrukcija, uz izdvajanje registara i memorijskih reči u koje je upisivano. Takav obim omogućava da se u narednim poglavljima detaljno objasni ceo put od izvornog teksta do promene stanja. Jednostavniji prikaz je svesno izabran pri projektovanju. Međutim, bez ispitivanja sa korisnicima ne može se tvrditi da je razumljiviji ili da korisnicima više pomaže u učenju od prikaza u RARS-u i Venus-u.

3. Teorijske osnove RISC-V arhitekture i simulacije

Za razumevanje simulatora potrebno je povezati značenje instrukcija sa promenama registara, memorije i toka programa. U ovom poglavlju objašnjeni su osnovni pojmovi RISC-V arhitekture i uloge komponenti koje obrađuju i izvršavaju asemblerski kod. Pravila arhitekture navedena su odvojeno od ograničenja izabranih za potrebe ovog rada.

3.1. RISC-V i pojam arhitekture skupa instrukcija

Arhitektura skupa instrukcija (engl. *Instruction Set Architecture*, ISA) određuje koje operacije procesor pruža programu, kako se one zadaju i kako utiču na stanje vidljivo programeru. Istu arhitekturu mogu da realizuju procesori različite unutrašnje organizacije. Zato razumevanje značenja instrukcije ne zahteva poznavanje načina na koji je procesor hardverski realizovan.

RISC-V je porodica arhitektura u kojoj se osnovni skup instrukcija za rad sa celim brojevima dopunjuje proširenjima. Oznaka RV32I odnosi se na osnovni skup sa 32-bitnim celobrojnim registrima, dok se, na primer, množenje i deljenje dodaju proširenjem M [1]. Razvijeni simulator podržava samo odabranih 15 instrukcija iz RV32I; to ne predstavlja potpunu implementaciju tog skupa.

RV32I pripada arhitekturama tipa *load-store*: podaci se između memorije i registara prenose posebnim instrukcijama, dok aritmetičke operacije koriste vrednosti iz registara ili konstantu zadatu u instrukciji [6]. Za sabiranje vrednosti iz memorije zato je potrebno prvo učitati podatke, zatim izvršiti sabiranje i, po potrebi, upisati rezultat nazad. Ovakav sled jasno razdvaja računanje od pristupa memoriji.

3.2. Registri, programski brojač i memorija

RV32I ima 32 registra označena sa x0–x31, širine 32 bita. Registar x0 uvek sadrži nulu, a pokušaj upisa ne menja njegovu vrednost [6]. Ostali registri čuvaju operande, rezultate i adrese; u zapisima instrukcija oznaka rd predstavlja odredišni registar, a rs1 i rs2 izvorne registre.

Isti niz od 32 bita može se tumačiti kao broj sa znakom ili bez znaka. Za brojeve sa znakom koristi se dvojni komplement: zapis 0xFFFFFFFF, na primer, predstavlja -1, dok kao broj bez znaka predstavlja 4294967295 [1] [6]. Pri sabiranju i oduzimanju zadržavaju se donja 32 bita rezultata, pa prekoračenje opsega ne proširuje registar; značenje prikazane vrednosti zavisi od izabranog tumačenja.

Programski brojač, skraćeno PC (engl. *Program Counter*), čuva adresu instrukcije. Osnovne RV32I instrukcije imaju dužinu od 4 bajta, pa se pri redoslednom izvršavanju prelazi na adresu veću za 4 [6]. Grane i skokovi mogu da promene taj redosled; u simulatoru PC između dva koraka označava instrukciju koja će se sledeća izvršiti.

Memorija je bajtovski adresibilna: svaka adresa označava jedan bajt, a memorijska reč od 32 bita zauzima 4 uzastopna bajta. U simulatoru je izabran *little-endian* raspored, pri kome se najmanje značajan bajt čuva na najnižoj adresi. Na primer, reč 0x12345678 upisana od adrese 0 raspoređuje se kao 0x78, 0x56, 0x34 i 0x12 na adresama 0, 1, 2 i 3; specifikacija izbor redosleda bajtova prepušta izvršnom okruženju [6].

Razvijeno rešenje ima 4096 bajtova memorije podataka i zahteva da adresa za LW ili SW bude deljiva sa 4. To je pravilo ovog simulatora, dok ponašanje pri neporavnom pristupu u RISC-V zavisi od izvršnog okruženja [6]. Instrukcije su u aplikaciji sačuvane odvojeno od podataka, pa se LW i SW ne koriste za čitanje ili menjanje samog programa.

3.3. Grupe podržanih instrukcija i pseudo-instrukcije

Instrukcije ADD i SUB sabiraju i oduzimaju vrednosti iz dva registra, dok ADDI sabira vrednost registra sa konstantom navedenom u samoj instrukciji (engl. *immediate*). Operacije AND, OR i XOR deluju na odgovarajuće bitove operanada. SLL i SRL pomeraju bitove ulevo ili logički udesno, pri čemu se oslobođena mesta popunjavaju nulama, a broj mesta pomeranja određuje donjih 5 bita drugog izvornog registra [6].

Logičko pomeranje udesno treba razlikovati od aritmetičkog, koje pri pomeranju čuva znak broja. Ako registar sadrži -8, odnosno 0xFFFFFFF8, SRL za jedno mesto daje 0x7FFFFFFC, što je 2147483644, a ne -4. Razlika nastaje zato što SRL na najviše mesto postavlja nulu, umesto da ponovi prethodni znak; aritmetičko pomeranje udesno nije deo podržanog skupa ovog simulatora [6].

LW učitava memorijsku reč u registar, a SW upisuje reč iz registra u memoriju. Adresa pristupa dobija se sabiranjem vrednosti baznog registra i pomeraja navedenog u instrukciji [6]. Tako LW x3, 8(x2) učitava u x3 reč sa adrese koja je za 8 bajtova veća od vrednosti u x2.

BEQ i BNE menjaju tok programa kada su vrednosti dva registra jednake, odnosno različite. BLT vrši poređenje brojeva sa znakom i grana se kada je prvi operand manji od drugog [6]. Odredište grane određuje se pomerajem u odnosu na adresu same instrukcije grananja, a ne sledeće instrukcije; u izvornom kodu ovog simulatora odredište se zadaje labelom.

JAL izvršava skok sa pomerajem u odnosu na PC, dok JALR računa odredište iz baznog registra i pomeraja, uz postavljanje najnižeg bita odredišne adrese na nulu. Obe instrukcije u rd upisuju povratnu adresu, odnosno adresu instrukcije koja sledi posle skoka [6]. Izbor x0 za rd odbacuje tu adresu i omogućava skok bez njenog čuvanja.

Pseudo-instrukcija je pogodniji zapis koji assembler prevodi u jednu ili više stvarnih instrukcija; procesor je ne izvršava kao posebnu operaciju. U ovom simulatoru podržano je 5 pseudo-instrukcija i svaka se prevodi u jednu instrukciju, prema Tabeli 2. Oznaka imm predstavlja konstantu, rs izvorni registar, a labela simboličko ime odredišta skoka.

Tabela 2. Prevođenje pseudo-instrukcija u razvijenom simulatoru

Pseudo-instrukcija	Prevod u razvijenom simulatoru
LI rd, imm	ADDI rd, x0, imm
MV rd, rs	ADDI rd, rs, 0
NOP	ADDI x0, x0, 0
J labela	JAL x0, labela
RET	JALR x0, 0(x1)

Instrukcija ADDI koristi 12-bitnu konstantu sa znakom, čiji je opseg od -2048 do 2047 [6]. Pošto se LI u ovom rešenju uvek prevodi u ADDI, i za nju važi isti opseg; nije podržano učitavanje proizvoljne 32-bitne konstante jednom pseudo-instrukcijom. RET koristi x1 kao registar povratne adrese, u skladu sa uobičajenom konvencijom poziva funkcija [6].

3.4. Uloga parsera, assemblera i simulatora

Obrada asemblerskog programa i njegovo izvršavanje predstavljaju različite zadatke. Parser u ovom radu prepoznaje strukturu izvornog teksta: labele, nazive instrukcija i njihove operande, uz uklanjanje komentara i praznih linija. Rezultat tog koraka još ne potvrđuje da instrukcija postoji niti da se može izvršiti sa navedenim operandima.

Labela daje ime određenom mestu u programu, ali sama po sebi nije instrukcija. Zato broj linije u editoru nije isto što i adresa instrukcije: komentari, prazne linije i labele napisane u zasebnom redu ne zauzimaju prostor u nizu izvršivih instrukcija. Veza sa izvornom linijom ipak se čuva, kako bi korisnik mogao da prati izvršavanje i pronađe prijavljenu grešku.

Assembler proverava podržanost instrukcija, vrste i broj operanada, opsege konstanti i ispravnost labela. U prvom prolazu povezuje labele sa adresama, a u drugom prevodi instrukcije i računa pomeraje, što omogućava skok i na labelu definisanu kasnije u tekstu. Ako je, na primer, instrukcija grananja na adresi 12, a odredište na adresi 4, potreban pomeraj iznosi -8 bajtova.

U uobičajenom postupku asembliranja izlaz uključuje mašinski kod, ali se u ovom radu formira niz objekata koji opisuju instrukcije. *Simulator* zatim čita pripremljene operande i menja registre, memoriju i PC, bez ponovnog tumačenja izvornog teksta. Ovim se razdvaja ispravnost zapisa od ponašanja pri izvršavanju, dok korisnički interfejs omogućava praćenje dobijenog stanja.

Uspešno asembliranje ne znači da će se program izvršiti bez greške, jer dozvoljenost nekih operacija zavisi od izračunate adrese ili trenutnog stanja. Na primer, SW x1, 2(x0) ima ispravan oblik i pomeraj koji staje u dozvoljeni opseg, ali pokušava upis na adresu 2, koja u ovom simulatoru nije poravnata za pristup reči. Takva greška otkriva se pri izvršavanju, a ne pri proveru zapisa.

4. Korišćene tehnologije i razvojni alati

Izbor tehnologija zasnovan je na potrebi da simulator radi u veb pregledaču, da se njegova logika može proveravati nezavisno od interfejsa i da implementacija ostane razumljiva. TypeScript je korišćen za programsku logiku, HTML i CSS za strukturu i izgled stranice, a Vite i Vitest za razvoj, pripremu aplikacije i testiranje.

4.1. TypeScript

TypeScript dopunjuje JavaScript sistemom tipova koji omogućava otkrivanje određenih grešaka pre izvršavanja programa. Proverava se, na primer, da li funkcija dobija argument odgovarajućeg tipa i da li objekat sadrži polje koje kod pokušava da koristi [7]. U simulatoru je to važno jer više komponenti razmenjuje podatke o operandima, instrukcijama i greškama.

Tipovima su opisani mogući oblici operandada, struktura pripremljene instrukcije i rezultati obrade programa. Registar, konstanta i memorijski operand predstavljeni su različitim varijantama istog tipa, pa kod najpre proverava vrstu operandada, a zatim pristupa odgovarajućim podacima. Takav opis čini očekivani oblik podataka vidljivim u samom kodu, umesto da se programer oslanja samo na dogovor ili komentar.

U podešavanjima TypeScript-a uključena je opcija *strict*, koja omogućava strožu proveru tipova tokom razvoja aplikacije. Dodatno su uključene provere neiskorišćenih promenljivih i parametara, kao i provera da funkcije vraćaju vrednost u svim granama u kojima se ona očekuje. Strože provere obuhvataju i rukovanje vrednostima *null* i *undefined* [7], što je korisno, na primer, kada model procesora još ne postoji ili traženi element stranice nije pronađen. Ipak, tipovi sami ne proveravaju da li je asemblerska konstanta u dozvoljenom opsegu ili da li je memorijska adresa poravnata; takve provere moraju biti deo logike aplikacije.

4.2. HTML, CSS i neposredan rad sa DOM-om

HTML (engl. *HyperText Markup Language*) određuje strukturu stranice: editor, dugmad, izbor primera i panele za registre i memoriju. CSS (engl. *Cascading Style Sheets*) određuje njihov raspored, boje, razmake i način isticanja važnih elemenata. U ovom projektu raspored je realizovan kombinacijom mehanizama *Grid* i *Flexbox*, a posebne klase se koriste za tekuću instrukciju, tačke prekida i promenjene vrednosti.

Objektni model dokumenta, odnosno DOM (engl. *Document Object Model*), predstavlja elemente stranice kao objekte kojima program može da pristupa i menja ih [8]. Klik na dugme povezuje se sa odgovarajućom funkcijom, a posle izvršene akcije ažuriraju se prikaz izvornog koda, registri, memorija i status. Za dinamički ispis teksta koristi se svojstvo *textContent*, tako da se uneti asemblerski kod prikazuje kao tekst, a ne tumači kao HTML.

Nije korišćena dodatna biblioteka ili radni okvir za korisnički interfejs, poput React-a. Za postojeći broj panela i kontrola neposredan rad sa DOM-om omogućio je jednostavno povezivanje korisničkih akcija sa simulatorom, bez uvođenja dodatnih pojmova i zavisnosti. Pravila izvršavanja ipak nisu smeštena u kod za prikaz: interfejs poziva model procesora i prikazuje njegove rezultate, ali ne određuje značenje instrukcija.

4.3. Vite i razvojno okruženje

Vite obezbeđuje lokalni razvojni server i podršku za obradu modula i TypeScript datoteka [9]. Komanda `npm run dev` pokreće server preko kog se aplikacija otvara u pregledaču, uz osvežavanje prikaza nakon izmena koda. Time se izmene mogu proveravati tokom razvoja, bez ručne pripreme svih datoteka posle svakog uređivanja.

Važno je razlikovati prevođenje TypeScript-a u JavaScript od provere tipova: Vite obavlja prevođenje, ali sam ne sprovodi tu proveru [9]. U projektu komanda `npm run typecheck` zato posebno pokreće TypeScript proveru, dok `npm run build` najpre proverava tipove, a tek potom priprema datoteke aplikacije za isporuku. Opcija `noEmit` sprečava da taj korak provere pravi zasebne JavaScript datoteke, pošto je njihova priprema poverena Vite-u.

Node.js se koristi za pokretanje razvojnih alata, a npm za upravljanje zavisnostima i izvršavanje projektnih komandi. Vite priprema TypeScript kod same aplikacije za izvršavanje u pregledaču tako što ga prevodi u JavaScript. Kada korisnik unese RISC-V program u editor, već pokrenuta aplikacija u pregledaču pretvara taj tekst u niz instrukcija koje izvršava model procesora. Korisnički RISC-V program se, dakle, ne šalje Vite serveru radi asembliranja ili izvršavanja.

4.4. Vitest i provera tipova

Vitest je alat za automatizovano izvršavanje testova povezan sa Vite razvojnim okruženjem [10]. Odabran je zato što se uklapa u postojeće razvojno okruženje i omogućava direktno korišćenje TypeScript modula u testovima. Pošto parser, assembler i model procesora ne zavise od DOM-a, njihovi testovi ne zahtevaju otvaranje aplikacije u pregledaču.

Test priprema ulaz, poziva funkciju koja se proverava i poredi dobijeni rezultat sa očekivanim. Na primer, test sabiranja izvršava kratak program i proverava vrednost određišenog registra, dok test memorije proverava upisane bajtove. Komanda `npm test` pokreće jednu proveru svih testova, a `npm run test:watch` ponavlja odgovarajuće testove tokom izmena; rezultati i granični slučajevi detaljno su predstavljeni u sedmom poglavlju.

Provera tipova i testovi imaju različite uloge. Ispravno tipiziran kod može da sadrži pogrešno računanje adrese ili rezultata instrukcije, što se otkriva testom ponašanja, dok provera tipova može da uoči pogrešnu upotrebu podataka pre pokretanja testa. Njihova kombinacija omogućava proveru različitih vrsta grešaka, ali ne zamenjuje ručnu proveru prikaza i rada korisničkih kontrola.

5. Projektovanje i implementacija sistema

Implementacija povezuje unos asemblerskog programa, njegovu proveru i izvršavanje sa prikazom stanja procesora. U ovom poglavlju objašnjeni su raspodela odgovornosti između modula, strukture podataka i postupci koji ostvaruju zadate funkcionalnosti. Isecci izvornog koda preuzeti su iz projekta, dok dijagrami prikazuju odnose između komponenti i tok obrade programa.

5.1. *Zahtevi i granice sistema*

Osnovni zahtevi bili su pisanje RISC-V programa u editoru, prevođenje u izvršive instrukcije, izvršavanje korak po korak, rad do tačke prekida i izvršavanje do završetka. Korisnik pritom treba da vidi registre, memoriju i liniju koja će se sledeća izvršiti. Iz ovih zahteva proistekla je potreba da se kroz sve faze obrade sačuva veza između izvorne linije i pripremljene instrukcije.

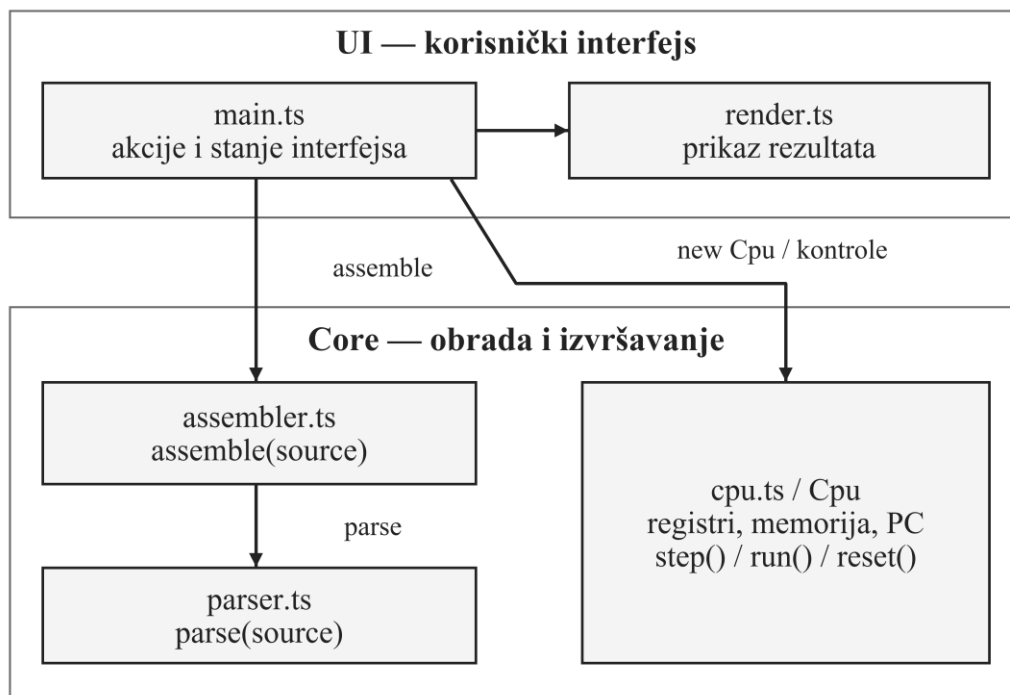
Podržani jezik namerno je ograničen na 15 instrukcija i 5 pseudo-instrukcija opisanih u prethodnim poglavljima. Izvorni program sadrži najviše jednu instrukciju po liniji, operande razdvojene zarezima, labele i komentare koji počinju znakom #. Model ima 32 registra i 4096 bajtova memorije podataka; nisu uključeni operativni sistem, periferije, binarno kodiranje instrukcija ni simulacija trajanja u taktovima.

Pored funkcionalnosti, pri projektovanju su bili važni razumljivost koda i mogućnost odvojenog testiranja. Zato pravila RISC-V instrukcija nisu smeštena u funkcije koje obrađuju klikove ili menjaju HTML elemente. Greške se vraćaju kao podaci koje interfejs može da prikaže, a neuspešno asembliranje ne sme da dovede do pokretanja delimično pripremljenog programa.

5.2. *Arhitektura i tok podataka*

Programska logika izdvojena je u direktorijum src/core, a korisnički interfejs u src/ui. Modul parser.ts prepoznaje zapis izvornog programa, assembler.ts proverava instrukcije i razrešava labele, a cpu.ts čuva stanje i izvršava pripremljeni program. Zajednički tipovi nalaze se u types.ts, čime je opis podataka koji prolaze između komponenti dostupan na jednom mestu.

Korisnički interfejs, skraćeno UI (engl. *User Interface*), povezuje ove komponente preko modula `main.ts`. On poziva assembler, pravi model procesora i obrađuje korisničke akcije, dok `render.ts` priprema prikaz programa, registara, memorije i grešaka. Podela komponenti i osnovni pozivi prikazani su na Slici 1; strelice označavaju pozive, a rezultati se vraćaju komponenti koja ih je zatražila.

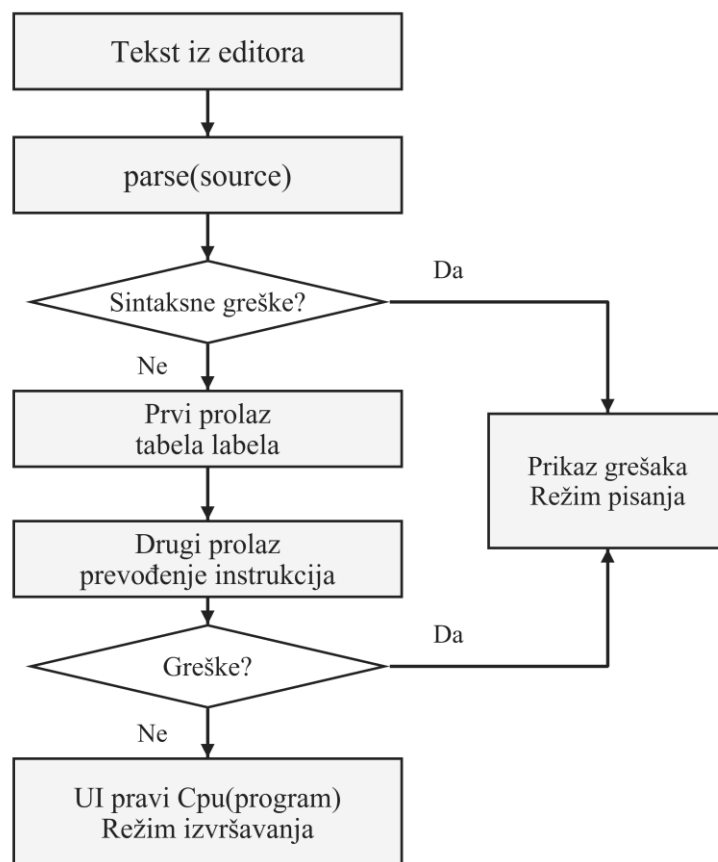


Slika 1. Podela komponenti i osnovni pozivi

Interfejs ne poziva parser zasebno: jedina ulazna tačka za obradu teksta jeste funkcija `assemble`. Ona unutar sebe pokreće parsiranje, pa pozivalac dobija ili pripremljeni program ili podatke o greškama. Time se redosled potrebnih provera ne ponavlja u kodu interfejsa.

Podaci o stanju pripadaju modelu procesora. Interfejs ih u postojećoj realizaciji čita i prikazuje, a izvršavanje pokreće metodama `step` i `run`; resetovanje se obavlja metodom `reset`. Ovako odvojena logika može da se proverava bez pravljenja stranice i njenih elemenata, dok promene rasporeda panela ne zahtevaju promenu značenja instrukcija.

Ukupan tok pripreme programa prikazan je na Slici 2. Sintaksne greške zaustavljaju obradu pre pravljenja tabele labela, dok se posle uspešnog parsiranja obavljaju oba prolaza asemblera. Tek kada nema prijavljenih grešaka, UI pravi novi objekat Cpu i prelazi u režim izvršavanja.



Slika 2. Tok obrade programa i prijave grešaka

5.3. Interna reprezentacija programa i stanja procesora

Izvorni tekst, rezultat parsera i izvršivi program nisu ista struktura podataka. Parser za svaku sadržajnu liniju pravi objekat `ParsedLine`, koji sadrži broj izvorne linije, opcionu labelu, naziv instrukcije i niz operandada. Operand nosi oznaku vrste `kind`, pa se registar, konstanta, labela i memorijski operand razlikuju već u rezultatu parsiranja.

Na primer, memorijski zapis `8(x2)` razlaže se na pomeraj 8 i broj baznog registra 2. Asembler zato ne mora ponovo da traži zagrade ili pretvara tekst u broj, već proverava da li takav operand odgovara konkretnoj instrukciji. Izvorni broj linije ostaje sačuvan i kada se uklone komentari i prazni redovi.

Sve pripremljene instrukcije imaju isti skup polja, opisan tipom `Instruction` u Segmentu izvornog koda 1. Polje `op` bira operaciju, `rd` označava odredište, `rs1` i `rs2` izvore, a `imm` konstantu ili pomeraj. Polja koja instrukcija ne koristi postavljaju se na nulu, što pojednostavljuje čitanje operanada u modelu procesora.

```
export interface Instruction {
  op: OpCode;
  rd: number;
  rs1: number;
  rs2: number;
  /** Konstanta; kod grana i skokova je PC-relativni pomeraj u bajtovima. */
  imm: number;
  /** Linija u editoru iz koje je instrukcija nastala (za `>` marker i breakpointe). */
  sourceLine: number;
}
```

Segment izvornog koda 1. Tip `Instruction` (`types.ts`)

Polje `sourceLine` nije deo RISC-V instrukcije, već dodatak potreban aplikaciji. Na osnovu njega se izvršiva instrukcija povezuje sa prikazom izvornog koda, porukom o grešci i tačkom prekida. Adresa instrukcije određuje se njenim mestom u nizu, u koracima od 4 bajta, pa broj reda u editoru ne mora da odgovara njenom indeksu u programu.

Registri su predstavljeni nizom `Int32Array` sa 32 elementa, a memorija nizom `Uint8Array` sa 4096 elemenata. To su tipizirani nizovi sa unapred određenim načinom čuvanja brojeanih vrednosti [11]. Upis u `Int32Array` daje 32-bitnu vrednost sa znakom, dok jedan element memorijskog niza predstavlja jedan bajt.

Objekat `DataView` koristi isti memorijski bafer i omogućava čitanje i upis reči uz izričit izbor redosleda bajtova [11]. CPU dodatno čuva programski brojač, skup registara u koje je upisivano i skup početnih adresa upisanih reči. Ti skupovi služe prikazu: pamte da je upis obavljen čak i kada je rezultat nula, umesto da se vidljivost određuje samo prema trenutnoj vrednosti.

5.4. Sintaksna analiza asemblerskog koda

Funkcija `parse` deli izvorni tekst na linije i svakoj zadržava originalni redni broj, počev od 1. Iz linije se uklanja deo od prvog znaka `#`, a zatim okolni razmaci. Ako posle toga nema sadržaja, linija se preskače; u suprotnom se proveravaju labela, naziv instrukcije i operandi.

Labela može da bude jedini sadržaj reda ili da prethodi instrukciji. Njeno ime sme da sadrži slova engleske abecede, cifre i donju crtu, ali ne sme da počne cifrom. Nazivi instrukcija pretvaraju se u velika slova, a zapis registra prihvata `x` i `X`; imena labela se ne menjaju, pa se pri njihovom povezivanju razlikuju velika i mala slova.

Operandi se razdvajaju zarezima. Prazan deo između dva zareza prijavljuje se kao greška, umesto da se tiho preskoči. Memorijski operand prepoznaje se po zagradama i dodatno proverava, dok se za ostale zapise redom pokušava prepoznavanje registra, konstante i imena labele.

Registar se ne prihvata samo zato što njegov naziv počinje slovom `x`. Posle prepoznavanja oblika proverava se broj, pa `x32` izaziva grešku, a ne postaje labela. Brojevi mogu da budu decimalni ili heksadecimalni, sa opcionim predznakom; proverava da li konstanta staje u određeni format instrukcije ostavljena je assembleru.

Parser prikuplja greške iz različitih linija i vraća ih zajedno sa uspešno pročitanim linijama. To korisniku omogućava da u jednom pokušaju vidi više problema, ali ne znači da se u svakoj neispravnoj liniji otkrivaju sve moguće greške. Kada parsiranje nije uspešno, assembler ne prelazi na semantičku proveru, čime se izbegavaju dodatne poruke zasnovane na nepotpuno protumačenom tekstu.

5.5. Dvoprolazni assembler i prevođenje pseudo-instrukcija

Funkcija `assemble` povezuje parsiranje, izgradnju tabele labela i prevođenje instrukcija. Njen kod dat je u Segmentu izvornog koda 2. Povratna vrednost sadrži program, tabelu labela i listu grešaka; program se koristi samo ako je lista grešaka prazna.

```
export function assemble(source: string): AssembleResult {
  const parsed = parse(source);
  if (parsed.errors.length > 0) {
    return { program: [], labels: new Map(), errors: parsed.errors };
  }

  const errors: AsmError[] = [];
  const labels = collectLabels(parsed.lines, errors);
  const program = translate(parsed.lines, labels, errors);

  return { program, labels, errors };
}
```

Segment izvornog koda 2. Povezivanje faza asembliranja (assembler.ts)

U prvom prolazu početna adresa iznosi nula. Pronađena labela vezuje se za tekuću adresu, a adresa se povećava za 4 samo ako linija sadrži instrukciju. Ponovno definisanje istog imena prijavljuje se kao greška; labela u posebnom redu može da označi sledeću instrukciju bez zauzimanja dodatnog mesta.

Drugi prolaz koristi već poznatu tabelu labela. Zato instrukcija grananja može da upućuje i unapred, na deo izvornog teksta koji se pojavljuje kasnije. Pomeraj se računa kao razlika određene adrese i adrese same grane ili skoka, dok se brojač adresa pomera i posle neispravne instrukcije, kako bi ostale poruke zadržale doslednu vezu sa izvornim rasporedom.

Podaci o pravim instrukcijama smešteni su u tabelu SPECS. Svaka stavka određuje očekivani format i tekstualni opis sintakse, a `buildInstruction` proverava broj i vrste operandata. Tako instrukcije sličnog oblika dele postupak provere, umesto da se za svaku posebno ponavlja ista logika.

Pre ove provere funkcija `expandPseudo` prevodi LI, MV, NOP, J i RET u odgovarajuće prave instrukcije. U ovoj implementaciji svako prevođenje daje tačno jednu instrukciju, pa prvi prolaz može unapred da dodeljuje adrese u koracima od 4. LI se posle prevođenja proverava kao ADDI i zato ne može da učitava konstantu van njegovog 12-bitnog opsega.

Proveravaju se i opsezi pomeraja u skladu sa korišćenim formatima. Pošto se određena grana u JAL-a zadaju labelama, a adrese instrukcija rastu za 4, dobijeni pomeraji prate taj raspored. JALR određuje zavisi od vrednosti registra pri izvršavanju, pa se njegov konačan cilj ne može u potpunosti proveriti tokom asembliranja.

Instrukcija SW zahteva posebnu pažnju pri raspoređivanju polja. Prvi operand predstavlja registar sa podatkom za upis, pa se smešta u `rs2`, a ne u `rd`; bazni registar iz memorijskog operanda postaje `rs1`. Kada postoje semantičke greške, deo instrukcija može biti formiran, ali UI odbacuje takav rezultat i ne pravi procesor.

5.6. Izvršavanje instrukcija, memorija i kontrola toka

Klasa Cpu dobija već pripremljen niz instrukcija. Svojstvo `currentInstruction` koristi PC da izračuna indeks u tom nizu, dok step izvršava jedan korak i vraća njegov ishod. Ako na tekućoj adresi nema instrukcije, vraća se status *halted*; inače se određuje podrazumevana naredna adresa i poziva obrada operacije.

Naredna adresa najpre se čuva u pomoćnom polju `nextPc`, umesto da se odmah promeni `pc`. Grana ili skok mogu da zamene tu adresu, a `pc` se ažurira tek posle uspešnog izvršavanja, kako pokazuje Segment izvornog koda 3. Time stara adresa ostaje dostupna za računanje odredišta i povratne adrese.

```
this.nextPc = this.pc + INSTRUCTION_SIZE;

const error = this.execute(instruction);
if (error !== null) {
  return { status: "error", message: error, line: instruction.sourceLine };
}

this.pc = this.nextPc;
return { status: "ok", line: instruction.sourceLine };
```

Segment izvornog koda 3. Ažuriranje programskog brojača u metodi step (cpu.ts)

Izvršavanje operacije određuje *switch* nad poljem `op`. Sabiranje i oduzimanje koriste vrednosti registara, a upis rezultata u `Int32Array` obezbeđuje svođenje na 32 bita. Za logičko pomeranje udesno koristi se operator `>>>`, ne `>>`, dok se broj mesta pomeranja ograničava maskom `0x1F`, u skladu sa pravilima instrukcija [6].

Upis u registar prolazi kroz pomoćnu metodu `setRegister`. Ona zanemaruje odredište `x0`, a za ostale registre upisuje rezultat i beleži broj registra u `usedRegisters`. Zahvaljujući tome, isti postupak štiti nulti registar i pri aritmetičkoj operaciji, učitavanju iz memorije i upisu povratne adrese.

LW i SW računaju adresu sabiranjem baze i pomeraja. Pre pristupa proverava se deljivost sa 4 i da li sva 4 bajta reči pripadaju memoriji, pa je poslednja dozvoljena početna adresa 4092. Čitanje i upis koriste `getInt32` i `setInt32` sa argumentom *true* za *little-endian* raspored; SW tek nakon uspešnog upisa dodaje adresu u `writtenWords`.

Grane BEQ, BNE i BLT menjaju `nextPc` samo kada je uslov ispunjen. BLT poredi vrednosti sa znakom, pošto su operandi pročitani iz `Int32Array`. Za JALR se nakon sabiranja baze i pomeraja briše najniži bit, ali i tako dobijeno odredište mora da zadovolji poravnanje na 4 bajta koje zahteva ovaj simulator.

Provera odredišta skoka obavlja se pre upisa povratne adrese. Ako je cilj neispravan, odredišni registar ostaje nepromenjen, a step vraća grešku bez pomeranja PC-a. Operandi su pročitani pre upisa rezultata, što je značajno kada JALR koristi isti registar kao bazu i odredište povratne adrese.

Adresa neposredno iza poslednje instrukcije dozvoljena je kao završetak programa. Uspešan step nad poslednjom instrukcijom vraća *ok*, a naredni poziv prepoznaje da više nema instrukcija; run završetak proverava i neposredno posle koraka. Ova razlika proizlazi iz toga što step opisuje pojedinačni korak, a run upravlja nizom koraka.

5.7. Kontrole izvršavanja i breakpointi

Metoda `run` koristi postojeću metodu `step` umesto zasebne implementacije instrukcija. U svakom prolazu najpre izvršava tekuću instrukciju, zatim proverava kraj programa i tačku prekida na sledećoj instrukciji. Taj redosled prikazan je u Segmentu izvornog koda 4.

```
for (let executed = 0; executed < maxSteps; executed++) {
  const result = this.step();
  if (result.status !== "ok") {
    return result;
  }

  const next = this.currentInstruction;
  if (next === null) {
    return { status: "halted" };
  }
  if (breakpoints?.has(next.sourceLine)) {
    return { status: "breakpoint", line: next.sourceLine };
  }
}
```

Segment izvornog koda 4. Petlja izvršavanja u metodi `run` (`cpu.ts`)

Provera tačke prekida posle koraka omogućava nastavak programa koji je već zaustavljen na toj tački. Novi *Run* izvršava tekuću instrukciju i tek potom traži sledeće zaustavljanje, umesto da odmah ponovo stane na istom mestu. Ako petlja ponovo dovede do iste linije, tačka prekida ponovo zaustavlja izvršavanje pre te instrukcije.

Tačke prekida čuva UI u skupu brojeva izvornih linija. Prilikom asembliranja pravi se i skup linija koje zaista nose instrukcije, pa klik na komentar ili samostalnu labelu ne postavlja tačku prekida. Dugme *Breakpoint* deluje na tekuću instrukciju, a klik na izvršivu liniju omogućava izbor drugog mesta.

Izvršavanje komandom *Run* je sinhrono, bez animacije pojedinačnih koraka. Jedan poziv ograničen je na 100000 instrukcija; ako se do tada ne dostigne kraj ili tačka prekida, vraća se poruka o mogućoj beskonačnoj petlji. To je zaštitno ograničenje, ne dokaz da je petlja zaista beskonačna, jer i ispravan program može zahtevati više koraka.

Reset postavlja registre, memoriju i PC na početne vrednosti i prazni evidenciju upisa, ali zadržava pripremljeni program. Tačke prekida ostaju sačuvane jer pripadaju UI stanju. Nasuprot tome, povratak u režim pisanja uklanja procesor i tačke prekida: izmena izvornog teksta može da promeni brojeve linija, pa prethodna mesta zaustavljanja više ne moraju imati isto značenje.

5.8. Povezivanje modela sa korisničkim interfejsom

UI ima režim pisanja i režim izvršavanja. U prvom su dostupni editor i izbor primera, a u drugom prikaz izvornog koda koji se ne može uređivati, uz kontrole izvršavanja. U njemu se vide i komentari i labele, zajedno sa brojevima linija, markerom tekuće instrukcije i tačkama prekida. Posle uspešnog asembliranja pravi se novi Cpu, dok prijavljene greške ostavljaju korisnika u režimu pisanja.

Metode `onStep` i `onRun` najpre kopiraju vrednosti registara, zatim pozivaju procesor i prosleđuju rezultat funkciji `applyResult`. Ona postavlja poruku o ishodu, beleži završetak ili grešku i ponovo iscertava prikaz. Registre za isticanje određuje poređenje stanja pre i posle akcije, a ne evidencija svake međupromene.

Zato se posle *Run-a* ističu registri čije se završne vrednosti razlikuju od početnih. Ako se registar tokom tog izvršavanja promeni i zatim vrati na istu vrednost, nije istaknut. Vidljivost registra je odvojeno pitanje: `usedRegisters` ga zadržava u panelu zato što je u njega upisivano, čak i kada trenutna vrednost iznosi nula.

Modul `render.ts` na osnovu stanja simulatora formira prikaz izvornog koda i panele sa vrednostima registara i memorije. Izvorni tekst prikazuje se preko `textContent`, pa se ne tumači kao HTML, a elementi se zamenjuju novim prikazom posle akcije. Za registre i memoriju prikazuju se heksadecimalna i decimalna vrednost; memorijske reči biraju se prema `writtenWords` i sortiraju po adresi.

Funkcija za obradu klikova povezana je sa zajedničkim kontejnerom koji obuhvata sve redove prikaza koda. Broj linije na koju je korisnik kliknuo određuje gde treba uključiti ili ukloniti tačku prekida. Ovakvo povezivanje ostaje važeće i kada se redovi prikaza ponovo naprave.

Primeri programa i kratka dokumentacija instrukcija izdvojeni su u posebne module. Oni dopunjuju interfejs, ali ne određuju pravila parsera, assemblera ili procesora. Time se razdvajaju prikaz, sadržaj namenjen korisniku i izvršna logika, a način korišćenja celog rešenja može zasebno da se prikaže u narednom poglavlju.

6. Funkcionalnosti i primer korišćenja

Rad aplikacije prikazan je kroz ugrađeni primer „Zbir brojeva 1-5”. Isti program koristi se za unos, izvršavanje pojedinačnih instrukcija, zaustavljanje na tački prekida i proveru konačnog rezultata. Na kraju je jedna instrukcija namerno izmenjena kako bi se pokazalo prijavljivanje greške pri asembliranju.

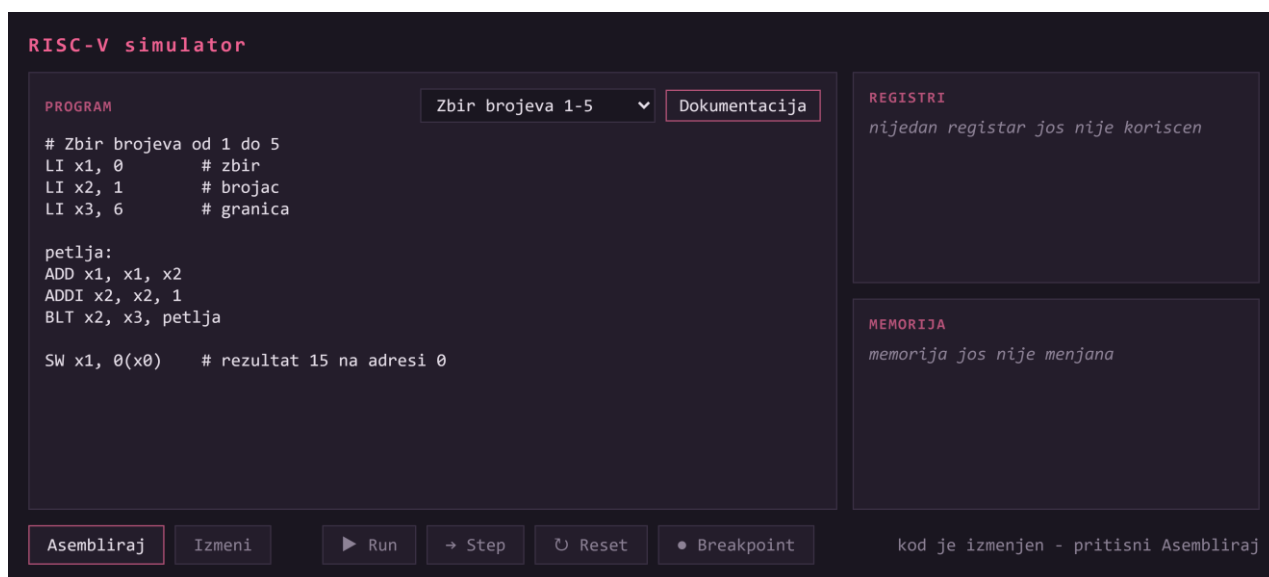
6.1. Izgled aplikacije i režimi rada

Levi deo radne površine namenjen je programu, a desni panelima REGISTRI i MEMORIJA. Ispod njih nalaze se kontrole izvršavanja i poruka o trenutnom stanju. Padajući meni omogućava izbor ugrađenih primera, dok dugme Dokumentacija otvara pregled podržanih instrukcija i njihovih formata.

U režimu pisanja korisnik uređuje tekst ili bira primer. Posle uspešnog asembliranja prelazi se u režim izvršavanja, u kome se izvorni kod prikazuje sa brojevima linija i oznakama, bez mogućnosti neposrednog uređivanja. Za povratak u editor koristi se dugme Izmeni.

6.2. Unos i asembliranje programa

Izborom stavke „Zbir brojeva 1-5” učitava se program prikazan na Slici 3. Registar x1 čuva zbir, x2 brojač, a x3 granicu 6. Petlja dodaje vrednosti od 1 do 5, nakon čega se rezultat instrukcijom SW upisuje na adresu 0.



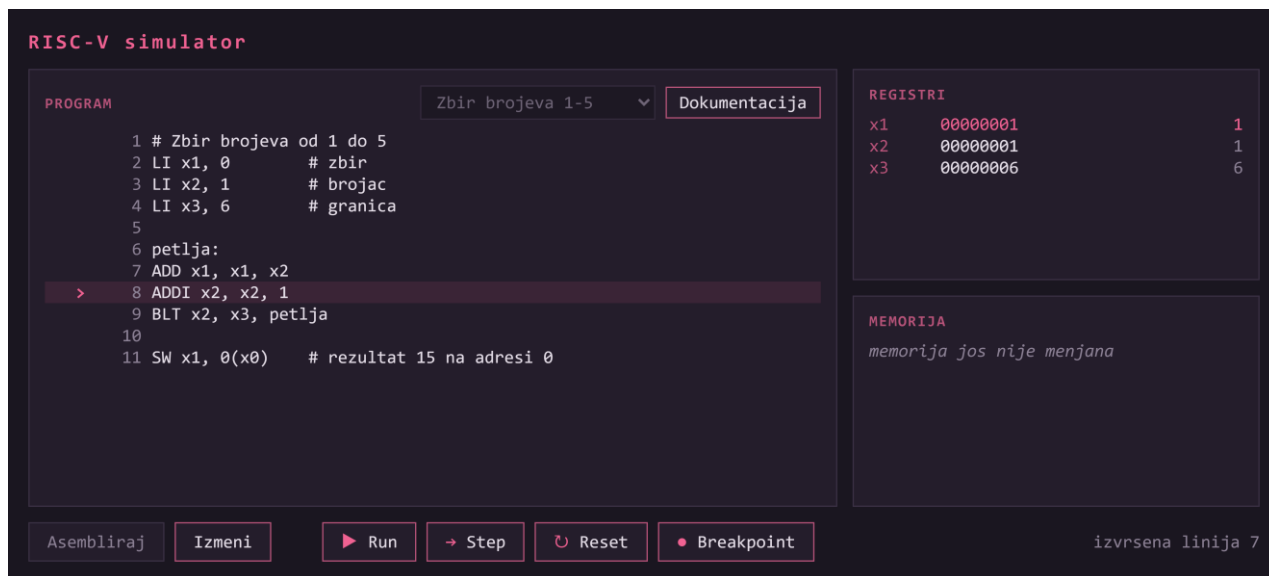
Slika 3. Primer sabiranja u režimu pisanja

Pritiskom na Asembliraj proverava se uneti program. Ako nema grešaka, omogućavaju se Step i Run, a marker > postavlja se na liniju 2, koja sadrži prvu instrukciju LI x1, 0. Registri i memorija još nisu prikazani kao korišćeni, pošto nijedna instrukcija nije izvršena.

6.3. Izvršavanje korak po korak

Prva tri pritiska na Step postavljaju x1 na 0, x2 na 1 i x3 na 6. Marker zatim pokazuje liniju 7, sa prvom instrukcijom tela petlje. Komentari, prazni redovi i samostalna labela petlja ne predstavljaju posebne korake izvršavanja.

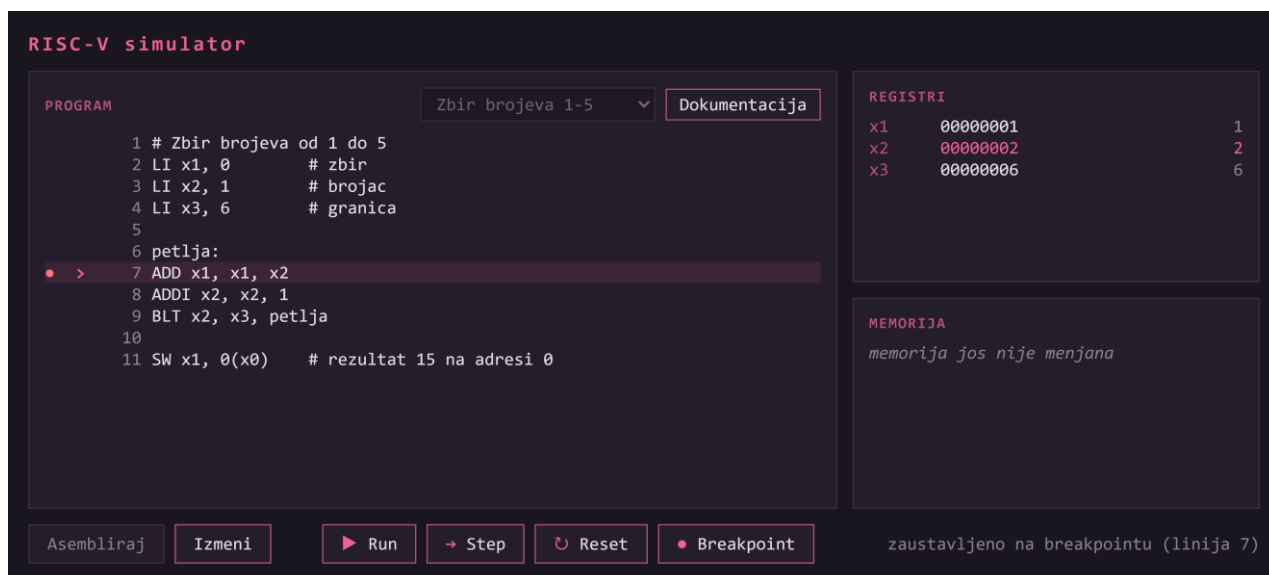
Četvrti *Step* izvršava `ADD x1, x1, x2` i menja zbir na 1. Stanje je prikazano na Slici 4: `x1` i `x2` imaju vrednost 1, `x3` ostaje 6, a memorija još nije menjana. Marker je na liniji 8 jer označava sledeću instrukciju, dok je promenjena vrednost registra `x1` vizuelno istaknuta.



Slika 4. Stanje posle četiri koraka izvršavanja

6.4. Run, breakpointi i Reset

Posle četvrtog koraka klikom na izvršivu liniju 7 postavlja se tačka prekida, označena crvenom tačkom. *Run* nastavlja od linije 8, povećava brojčac i izvršava grananje nazad ka petlji. Program se zaustavlja pre narednog sabiranja: `x1` je 1, `x2` je 2, a `x3` je 6, kao na Slici 5.



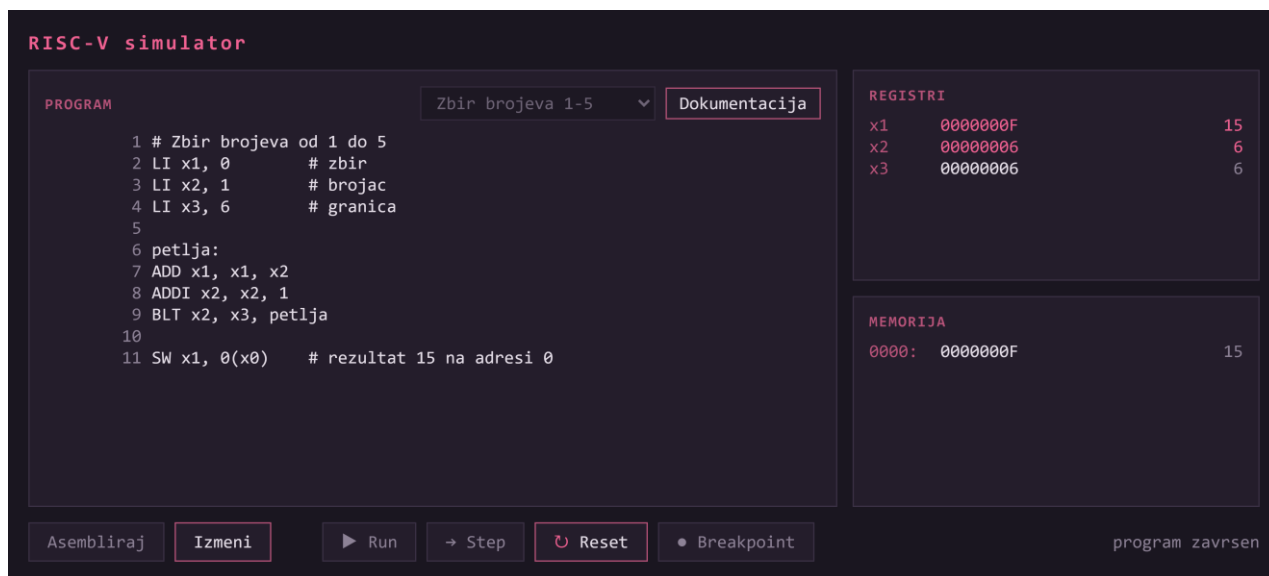
Slika 5. Zaustavljanje pre drugog sabiranja u petlji

Ponovni *Run* izvršava instrukciju na kojoj je program zaustavljen i nastavlja do sledećeg nailaska na tačku prekida. Tada su `x1` i `x2` jednaki 3, dok `x3` ostaje 6. Tačka se uklanja ponovnim klikom na liniju ili dugmetom *Breakpoint* kada je marker na toj liniji, nakon čega *Run* može da dovrši program.

Reset vraća registre i memoriju na nulu, a marker na početnu instrukciju. Prikaz upisivanih registara i memorijskih reči tada se prazni, ali učitani program i tačke prekida ostaju sačuvani. Izmeni se razlikuje od *Reset-a*: vraća uređivanje teksta i uklanja prethodno stanje izvršavanja i tačke prekida.

6.5. Prikaz registara, memorije i grešaka

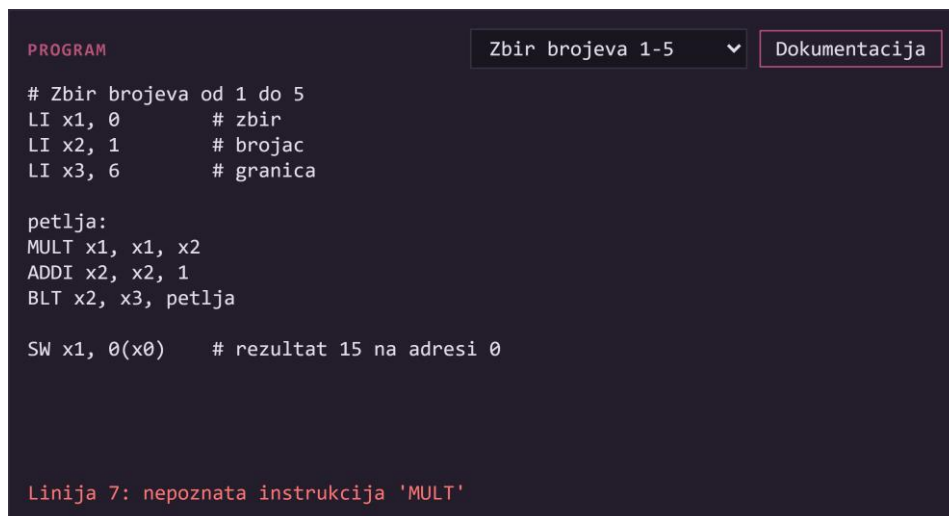
Po završetku programa x1 sadrži zbir 15, a x2 i x3 imaju vrednost 6. Reč na adresi 0000 prikazana je heksadecimalno kao 0000000F i decimalno kao 15, što potvrđuje upis rezultata. Završno stanje prikazano je na Slici 6; marker više ne označava instrukciju, a *Step* i *Run* su onemogućeni.



Slika 6. Konačan rezultat u registrima i memoriji

Primer pokazuje zašto su korisna oba zapisa vrednosti. Heksadecimalni zapis omogućava praćenje sadržaja 32-bitne reči, dok se iz decimalnog neposredno čita rezultat sabiranja. Memorijski panel prikazuje samo upisanu reč, a ne ceo prostor od 4096 bajtova.

Za prikaz greške program se vraća u editor i naziv ADD u liniji 7 menja u MULT. Ponovno asembliranje prijavljuje poruku „Linija 7: nepoznata instrukcija 'MULT'”, prikazanu na Slici 7. Aplikacija ostaje u režimu pisanja i ne dozvoljava izvršavanje dok se greška ne ispravi.

The image shows a screenshot of an assembly editor interface. At the top, there is a title bar with the text "PROGRAM" on the left, a dropdown menu showing "Zbir brojeva 1-5" in the center, and a button labeled "Dokumentacija" on the right. The main area of the editor contains assembly code. The code starts with a comment "# Zbir brojeva od 1 do 5" followed by three "LI" instructions: "LI x1, 0 # zbir", "LI x2, 1 # brojac", and "LI x3, 6 # granica". Then, a label "petlja:" is followed by three instructions: "MULT x1, x1, x2", "ADDI x2, x2, 1", and "BLT x2, x3, petlja". Finally, there is a "SW x1, 0(x0) # rezultat 15 na adresi 0" instruction. At the bottom of the editor, a red error message reads "Linija 7: nepoznata instrukcija 'MULT'".

```
PROGRAM                                Zbir brojeva 1-5 ▼ Dokumentacija

# Zbir brojeva od 1 do 5
LI x1, 0      # zbir
LI x2, 1      # brojac
LI x3, 6      # granica

petlja:
MULT x1, x1, x2
ADDI x2, x2, 1
BLT x2, x3, petlja

SW x1, 0(x0)  # rezultat 15 na adresi 0

Linija 7: nepoznata instrukcija 'MULT'
```

Slika 7. Greška nakon izmene naziva instrukcije

Greška može da nastane i tokom izvršavanja. Ako se u zasebnom programu unesu LI x1, 15 i zatim SW x1, 2(x0), asembliranje uspeva, ali *Run* prijavljuje da adresa 0x002 nije poravnata na 4 bajta. Registar x1 tada sadrži 15, memorija ostaje neizmenjena, a poruka upućuje na drugu liniju programa, gde je izvršavanje prekinuto.

7. Testiranje i analiza rezultata

Provera simulatora obuhvata obradu asemblerskog teksta, izvršavanje instrukcija i rad korisničkih kontrola. Cilj nije samo da izabrani primer daje očekivani zbir, već i da se neispravni ulazi odbace i granični slučajevi obrade na predviđen način. U ovom poglavlju opisani su postojeći testovi, postupak provere i granice zaključaka koji se mogu izvesti iz rezultata.

7.1. Strategija i organizacija testiranja

Automatski testovi organizovani su u tri datoteke: `parser.test.ts`, `assembler.test.ts` i `cpu.test.ts`. Svaki test priprema ulaz ili početno stanje, pokreće odgovarajuću funkciju i poredi dobijeni rezultat sa očekivanim. Ista logika može se proveravati više puta bez korisničkog interfejsa, jer `parser`, `assembler` i model procesora ne zavise od elemenata veb stranice.

Testovi su grupisani prema komponenti koju prvenstveno proveravaju, ali nisu svi potpuno izolovani. Većina CPU testova koristi stvarni assembler za pripremu programa, pa se istovremeno proverava i njihovo povezivanje. U pojedinim testovima registri se neposredno postavljaju na potrebne vrednosti, na primer kada početna vrednost ne može da se učita ograničenom pseudo-instrukcijom `LI`.

Završna provera za potrebe ovog rada izvršena je 29.09.2026. u Windows okruženju, uz `Node.js` 24.19.0 i `Vitest` 5.0.0. Testovi su pokrenuti projektnom komandom `npm test`, a provera tipova i priprema aplikacije komandom `npm run build`. Korišćene verzije `TypeScript`-a i `Vite`-a bile su 7.0.2 i 8.3.0.

7.2. Testiranje parsera i assemblera

Testovi parsera proveravaju prazan ulaz, uklanjanje komentara, prepoznavanje registara, konstanti, labela i memorijskih operanada. Obuhvaćeno je i očuvanje izvornog broja linije, koje je potrebno za poruke o greškama i praćenje izvršavanja. Odvojeno se proverava da parser može da pročita sintaksno ispravan naziv `MULT`, jer odluka o tome da li instrukcija postoji pripada assembleru.

Neispravni primeri uključuju registar `x99`, bazni registar `x32`, labelu koja počinje cifrom, višak zareza i memorijski operand bez pomeraja. Test sa više pokvarenih linija proverava da se greške prijavljuju sa brojevima 2, 3 i 4, dok ispravna linija ostaje u rezultatu parsiranja. Pored broja grešaka, proveravaju se njihove izvorne linije i karakteristični delovi poruka.

Assemblerski testovi porede polja nastale instrukcije sa očekivanim vrednostima. Posebno se proveravaju prevođenje svih 5 pseudo-instrukcija, raspored izvora kod `SW`, labele ispred i iza mesta upotrebe i pozitivni i negativni pomeraji. Vrednosti -2048 i 2047 prihvataju se kao granice 12-bitnog opsega, a neispravni memorijski pomeraji -2049 i 2048 odbacuju se.

Za grane su provereni krajnji dostižni pomeraji -4096 i 4092, kao i odbacivanje pomeraja 4096. Pozitivna granica 4092 proizlazi iz rasporeda instrukcija na adresama deljivim sa 4 u ovom simulatoru. Testovi obuhvataju i duplirane i nedefinisane labele, pogrešne operande i prekid asembliranja kada parser već prijavi grešku.

7.3. Testiranje modela procesora i graničnih slučajeva

Testovi procesora proveravaju rezultate aritmetičkih i bitskih operacija, nulti registar, promenu `PC`-a i završetak programa. U testu preliivanja početna vrednost 2147483647 neposredno se upisuje u registar, a dodavanje 1 daje -2147483648. Za `SRL` se proverava da pomeranje vrednosti -8 udesno za jedno mesto ne daje -4, čime se razlikuje logičko pomeranje od aritmetičkog.

Memorijski testovi obuhvataju upis i ponovno čitanje reči, očuvanje negativne vrednosti i *little-endian* raspored bajtova. Upis 0x12345678 mora da ostavi bajtove 0x78, 0x56, 0x34 i 0x12 na uzastopnim adresama. Prihvatljiv je pristup reči od adrese 4092, dok se neporavnata adresa 2, negativna adresa i adresa 4096 prijavljuju kao greške.

Kod grananja se proveravaju i ispunjeni i neispunjeni uslovi, a kod JAL i JALR odredište i povratna adresa. Poseban slučaj je JALR koji za bazu i povratnu adresu koristi isti registar: cilj mora da bude izračunat pre upisa nove vrednosti. Provereno je i da neispravan skok ne menja registar povratne adrese.

Jedan regresioni test proverava završetak tačno na granici dozvoljenog broja koraka. Ranije uočena greška u tom slučaju dovela je do prijave moguće beskonačne petlje iako je program završen. Ključni deo provere prikazan je u Segmentu izvornog koda 5; pomoćna funkcija `cpuFor` asembliira navedeni program i pravi procesor, a očekuju se status *halted* i rezultat 12.

```
const cpu = cpuFor(["LI x1, 5", "LI x2, 7", "ADD x3, x1, x2"].join("\n"));
expect(cpu.run({ maxSteps: 3 }).status).toBe("halted");
expect(cpu.registers[3]).toBe(12);
```

Segment izvornog koda 5. Provera završetka na granici broja koraka (cpu.test.ts)

Drugi test prekida petlju koja bezuslovno skače na samu sebe, uz granicu od 1000 koraka. Testovi tačaka prekida proveravaju zaustavljanje pre označene instrukcije, nastavak sa nje i slučaj kada *Step* dovede program na liniju sa tačkom prekida. Tako se proverava dogovoreno ponašanje komandi, ne samo pojedinačnih RISC-V instrukcija.

7.4. Ručna provera korisničkog interfejsa

Testovi logike ne potvrđuju da se marker, dugmad i vrednosti pravilno prikazuju na ekranu. Zato je proverom interfejsa obuhvaćeno povezivanje korisničkih akcija sa prikazanim stanjem: prelazak između režima, aktivnost kontrola, mesto zaustavljanja i prikaz poruka. Vizuelno su pregledane i slike prikazane u šestom poglavlju.

Pri pripremi tih slika scenario sabiranja brojeva od 1 do 5 izvršen je u pregledaču Microsoft Edge pomoću pomoćne Playwright skripte [12]. Provereni su koraci izvršavanja, ponovni *Run* sa *breakpointa*, očuvanje tačke prekida posle *Reset-a* i konačan upis vrednosti 15. Dodatno su provereni greška za *MULT*, neporavnan upis i otvaranje i zatvaranje dokumentacije.

Ova pomoćna automatizacija služila je proveriti demonstracije i pripremi slika. Nije uvedena kao trajni test paket korisničkog interfejsa i ne ulazi u broj postojećih Vitest testova. Vizuelni pregled i provera odabranih akcija ne zamenjuju sistematsko ispitivanje u različitim pregledačima niti proveru upotrebljivosti sa korisnicima.

7.5. Rezultati i ograničenja provere

U izvršenoj proveri prošlo je svih 113 testova, bez neuspešnih ili preskočenih testova. Raspodela po komponentama prikazana je u Tabeli 3. Navedeni brojevi predstavljaju rezultate stvarnog pokretanja postojećeg test paketa, a ne procenu na osnovu broja datoteka ili linija koda.

Tabela 3. Rezultati automatskih testova

Oblast	Broj testova	Uspešni	Neuspešni
Parser	24	24	0
Asembler	31	31	0
Model procesora	58	58	0
Ukupno	113	113	0

Uspešno je završena i komanda `npm run build`, koja prvo pokreće TypeScript proveru bez generisanja izlaznog koda, a zatim Vite pripremu aplikacije. To potvrđuje da proverena verzija prolazi proveru tipova i može da se pripremi za pokretanje, ali samo po sebi ne dokazuje ispravnost simulacije. Za ponašanje instrukcija relevantni su testovi sa konkretnim ulazima i očekivanim rezultatima.

Prolazak svih testova ne znači da je proverena svaka moguća kombinacija operanada i stanja. Nije merena pokrivenost koda, niti je sprovedeno automatsko poređenje rezultata sa drugim simulatorom. Testovi zato pružaju proverljive primere ispravnog ponašanja i zaštitu od ponavljanja poznatih grešaka, ali nisu formalni dokaz potpune ispravnosti.

Nisu sprovedena ni uporedna merenja performansi ni istraživanje koje bi pokazalo koliko aplikacija pomaže u učenju. Takve zaključke trebalo bi zasnovati na posebno pripremljenim merenjima i radu sa korisnicima. Rezultati ovog rada odnose se na funkcionalnost podržanog podskupa instrukcija i ispitane načine korišćenja aplikacije.

8. Zaključak

U okviru ovog rada razvijena je veb aplikacija za pisanje, asembliranje i izvršavanje programa za odabrani podskup RISC-V instrukcija. Korisnik može da izvršava program korak po korak, da nastavi izvršavanje do tačke prekida ili do završetka i da prati sadržaj registara i memorije. Time su ostvareni postavljeni ciljevi u dogovorenom obimu od 15 instrukcija i 5 pseudo-instrukcija, uz mogućnost ponovnog pokretanja programa i prijavljivanje grešaka.

Arhitektura aplikacije zasniva se na razdvajanju obrade izvornog koda, modela procesora i korisničkog interfejsa. Parser izdvaja strukturu teksta, assembler proverava instrukcije i razrešava labela, a procesor izvršava pripremljeni program. Takva podela omogućila je da se pravila izvršavanja proveravaju nezavisno od prikaza, dok je veza sa izvornim linijama sačuvana za poruke o greškama, marker i tačke prekida.

Posebnu pažnju zahtevali su slučajevi u kojima naizgled mala razlika u redosledu operacija menja rezultat. Među njima su računanje pomeraja u odnosu na adresu same instrukcije, čuvanje povratne adrese kod skokova i nastavak sa tačke prekida. Odvojeno određivanje naredne vrednosti programskog brojača i provera odredišta pre upisa rezultata pomogli su da se ova pravila dosledno sprovedu.

Provera nije bila ograničena na primere koji se uspešno završavaju. Postojeći testovi obuhvataju i neispravne operande, granice konstanti, neporavnate i nedozvoljene adrese, 32-bitno prelićvanje i posebne slućajevne skokova. Pri završnoj proveru prošlo je svih 113 testova, a uspešno su završene i provera TypeScript tipova i priprema aplikacije za pokretanje; ovi rezultati potvrđuju ponašanje u ispitanim slućajevima, ali nisu dokaz potpune ispravnosti.

Primer sabiranja brojeva od 1 do 5 pokazao je kako korisnik može da poveže pojedinaćnu instrukciju sa promenom stanja. Izvršavanje koracima otkriva promenu zbira i brojaća, taća prekida omogućava ponovljeno zaustavljanje na ulazu u petlju, a završni prikaz memorije potvrđuje upis rezultata. Prikaz vrednosti u heksadecimalnom i decimalnom obliku pritom omogućava da se posmatraju i njihov zapis i brojćano znaćenje.

Razvijeno rešenje ima jasno odrećena ogranićenja. Nije implementiran ceo RV32I skup, a program se ne prevodi u binarni mašinski kod, već u internu reprezentaciju koju izvršava simulator. Memorija podataka ogranićena je na 4096 bajtova i odvojena od programa, dok obrada periferija, operativnog sistema i unutrašnje hardverske organizacije procesora nije deo modela.

Ogranićenja postoje i u upravljanju izvršavanjem. *Run* radi sinhrono i prikazuje stanje po završetku akcije, a zadati broj dozvoljenih koraka sprećava neogranićeno trajanje jednog poziva. Aplikacija ne ćuva istoriju svih prethodnih stanja, pa ponovno razmatranje ranije instrukcije zahteva *Reset* i novo izvršavanje do željenog mesta.

Dalji razvoj mogao bi najpre da obuhvati proširenje skupa instrukcija i podršku za dodatne oblike asemblerskog zapisa. Takve izmene zahtevale bi dopunu parsera, assemblera i procesora, ali i odgovarajuće testove, kako se postojećim programima ne bi promenilo ponašanje. Kod složenijeg prevoćenja pseudo-instrukcija trebalo bi ponovo razmotriti i računanje adresa, jer sadašnje rešenje pretpostavlja da svaka zauzima mesto jedne instrukcije.

Poseban pravac unaprećenja bio bi generisanje i prikaz mašinskog koda. Time bi korisnik mogao da poveže tekstualni oblik instrukcije sa njenim bitskim poljima, što trenutna aplikacija ne prikazuje. Ova mogućnost predstavljala bi proširenje postojećeg assemblera, a ne samo promenu izgleda interfejsa, pa bi zahtevala nezavisnu proveru ispravnosti kodiranja.

Sa strane korisničkog interfejsa korisni dodaci bili bi isticanje sintakse, učitavanje i čuvanje programa i mogućnost vraćanja jednog koraka. Postupno izvršavanje uz osvežavanje prikaza i mogućnost pauziranja olakšalo bi praćenje dužih programa u odnosu na sadašnji sinhroni *Run*. Pri takvim izmenama bilo bi važno zadržati jasnu razliku između režima pisanja i izvršavanja, kao i dosledno ponašanje tačaka prekida.

Pouzdanost bi se mogla dodatno proveravati većim brojem graničnih i automatski generisanih programa. Za deo instrukcija koji različiti alati zajednički podržavaju korisno bi bilo upoređivati rezultate sa drugim simulatorom, uz uvažavanje razlika u sintaksi i izvršnom okruženju. Trajni testovi korisničkog interfejsa mogli bi da dopune postojeće testove logike, naročito pri promenama kontrola i načina prikaza stanja.

Pre primene u nastavi trebalo bi posebno ispitati koliko aplikacija pomaže korisnicima da razumeju izvršavanje asemblerskih programa. Takva provera mogla bi da koristi unapred pripremljene zadatke i posmatra tačnost odgovora, vreme potrebno za rešavanje i teškoće pri korišćenju interfejsa. Bez takvog istraživanja može se govoriti o mogućnosti upotrebe alata za demonstraciju, ali ne i o dokazanom poboljšanju učenja.

Glavni doprinos rada jeste povezana i proverena realizacija editora, asemblera i simulatora čije se ponašanje može pratiti od izvornog teksta do promene registara i memorije. Njena vrednost nije u većem broju funkcija u odnosu na postojeće alate, već u jasnom obimu, razumljivoj implementaciji i mogućnosti detaljnog objašnjenja svakog koraka. Takvo rešenje ispunjava početnu namenu i pruža osnovu za dalje proširivanje i proveru.

Reference

- [1] RISC-V International. *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture*, izdanje 20260120, *Introduction: RISC-V ISA Overview*. <https://docs.riscv.org/reference/isa/v20260120/unpriv/intro.html> [pristupljeno 28.09.2026.].
- [2] TheThirdOne/rars. RARS — RISC-V *Assembler and Runtime Simulator*, opis projekta. <https://github.com/TheThirdOne/rars> [pristupljeno 28.09.2026.].
- [3] TheThirdOne/rars. *Interactive Debugging Features*, dokumentacija projekta. <https://github.com/TheThirdOne/rars/blob/master/src/help/Debugging.html> [pristupljeno 28.09.2026.].
- [4] kvakil/venus. Venus, opis projekta. <https://github.com/kvakil/venus> [pristupljeno 28.09.2026.].
- [5] Venus. Veb interfejs simulatora. <https://venus.kvakil.me/> [pristupljeno 28.09.2026.].
- [6] RISC-V International. *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture*, izdanje 20260120, *RV32I Base Integer Instruction Set, Version 2.1*. <https://docs.riscv.org/reference/isa/v20260120/unpriv/rv32.html> [pristupljeno 28.09.2026.].
- [7] TypeScript. *The TypeScript Handbook: The Basics*. <https://www.typescriptlang.org/docs/handbook/2/basic-types.html> [pristupljeno 28.09.2026.].
- [8] MDN Web Docs. *Document Object Model (DOM)*. https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model [pristupljeno 28.09.2026.].
- [9] Vite. *Features*. <https://vite.dev/guide/features.html> [pristupljeno 28.09.2026.].
- [10] Vitest. *Getting Started*. <https://vitest.dev/guide/> [pristupljeno 28.09.2026.].
- [11] MDN Web Docs. *JavaScript typed arrays*. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Typed_arrays [pristupljeno 29.09.2026.].
- [12] Playwright. *Playwright Library*, dokumentacija za Python. <https://playwright.dev/python/docs/library> [pristupljeno 29.09.2026.].